```
//netlink message header base address
iov.iov_base = (void *)nlh;

//netlink message length
iov.iov_len = nlh->nlmsg_len;

//define the message header for message
//sending
struct msghdr msg;

msg.msg_name = (void *)&dest_addr;

msg.msg_namelen = sizeof(dest_addr);

msg.msg_iov = &iov;

msg.msg_iovlen = 1;
//send the message
sendmsg(fd, &msg, 0);
```

Unicast receive example: In case of the receiver, first the Netlink socket will be created using a socket API, as it was in the case of the sender.

Then, like the sender, the receiver will bind its socket with the unique address, which will be the same as in the case of the sender. *src_addr.nl_pid* should be initialised as follows:

```
//receiver address or id
src_addr.nl_pid = 2;
```

dest_addr will be used to receive the data which does not need to be initialised. In case of *nlmsghdr*, this structure is just cleared as follows:

```
memset(nlh, 0, NLMSG_SPACE(NLINK_MSG_PAYLOAD));
```

The rest of the code will be similar to the sender's code, but instead of the *sendmsg* API, the *recvmsg* API will be used as follows:

```
recvmsg(fd, &msg, 0);
```

This API will block until the message is received, after which the *nlmsghdr* variable *nlh* will get updated with the message header and payload, where the latter can be accessed as NLMSG_DATA(nlh) which will be a pointer to the payload.

Process-to-process multicast communication

In case of receivers for multicast communication, the *sockaddr_nl* structure should be initialised as follows:

```
struct sockaddr_nl src_addr;
//initialize the protocol as Netlink family
src_adr.nl_family = AF_NETLINK;

//assign the unique id to each application, here
//2 is assigned for example
src_addr.nl_pid = 2;

//assign multicast addresses on which the process
//wants to listen, for example all the process
//wants to listen on multicast address 3 and 5
src_addr.nl_groups = 1<<3 | 1<<5;
```

The rest of the code is similar to the unicast receiver code.

In case of a sender for multicast communication, the destination *sockaddr_nl* structure should be initialised as follows:

```
struct sockaddr_nl dest_addr;

//initialize the protocol as Netlink family
dest_addr.nl_family = AF_NETLINK;

//suppose process wants to send multicast
//message to all process with multicast
//group 3
dest_addr.nl_groups = 1<<3;
```

Kernel Netlink implementation

The kernel space API for Netlink is different from that for user space. To create a Netlink socket in the kernel, the following API is used:

```
struct sock* netlink_kernel_create(int unit,
void (*input)(struct sock *sock, int len));
```

In the above code:

- *unit* is the protocol type, which is defined in *include/uapi/linux/netlink.h*; for example, NETLINK_GENERIC.
- *input* is the function pointer, which is called when the application sends data to the kernel with a unit type protocol.

So to create a Netlink socket in the kernel module with NETLINK_GENERIC protocol type, *netlink_kernel_create* is called as follows:

```
struct sock* nlink

nlink = netlink_kernel_create(NETLINK_GENERIC, receive_func);

receive_func for example, is implemented as follows:

void receive_func(struct sock *sock, int len)
{
struct sk_buff *buffer;
struct nlmsghdr *nlh;
```